Smart Tractor Monitor: An STM32-RTOS and Raspberry Pi Architecture for Real-Time Telemetry and Control of Agricultural Vehicles

Juan José Jáuregui Barba, Rosendo De Los Ríos Moreno, Jordan Arturo Palafox Salinas, Marcelo Marcos Flores
Tecnológico de Monterrey — System Design on Chip, in collaboration with John Deere
{A00836722, A01198515, A00835705, A01722788}@tec.mx

***Abstract*—This paper presents Smart Tractor Monitor, an embedded telemetry and control system developed with John Deere to demonstrate real-time monitoring and driving assistance for agricultural tractors. An STM32F103RBT6 microcontroller reads an accelerometer-based throttle input and a matrix keypad, runs a four-task rate-monotonic (RMS) scheduled RTOS to model engine speed, vehicle speed, and gear state, and reports the result on a local LCD and over a UART/FTDI link to a Raspberry Pi 3 Model B+. On the host side, a Python application originally plotted the incoming stream with Matplotlib, which produced approximately one minute of display latency and made the system unusable for time-critical operations such as planting and harvesting. We diagnose the sources of this latency — a low UART baud rate, an unthreaded acquisition/render pipeline, and a heavyweight plotting backend — and resolve them by moving to PyQtGraph with a multithreaded producer/consumer pipeline, reducing the pipeline to near real time. We further add a bidirectional Tkinter dashboard that lets an operator command torque, braking, and direction from the Raspberry Pi back to the STM32, in addition to the physical keypad on the microcontroller side. The resulting system validates a low-cost, dual-processor architecture for real-time agricultural machine monitoring and remote actuation.**

***Index Terms*—STM32, real-time operating systems, rate-monotonic scheduling, Raspberry Pi, embedded Linux, UART, FTDI, PyQtGraph, precision agriculture**

I. Introduction

Modern agriculture increasingly depends on the integration of embedded electronics into heavy machinery to raise operational efficiency, reduce fuel consumption and mechanical wear, and lower the cognitive load on the operator. As part of a collaboration with John Deere, this project develops a smart driving-assistance and monitoring system for tractors that couples an STM32F103 microcontroller with a Raspberry Pi single-board computer. The STM32F103 measures motor speed through an accelerometer input, manages simulated gear changes, and exposes the resulting state on a local LCD; the data is then relayed over a UART/FTDI serial bridge to a Raspberry Pi, which is responsible for real-time visualization and for issuing control commands back to the microcontroller.

Existing commercial solutions in precision agriculture — GPS-guided auto-steer, remote telemetry, and fleet-management platforms — focus primarily on centimeter-level positioning and offline reporting. This project instead targets the tighter control loop: a low-cost, locally closed feedback path between an accelerometer-driven throttle model and a live dashboard, capable of both visualizing engine/vehicle state and actuating torque, braking, and direction commands with minimal latency.

A. Objectives and Contributions

The project pursued a single functional goal — real-time, bidirectional monitoring and control of a simulated tractor drivetrain — decomposed into the following contributions:

1) A four-task, rate-monotonic scheduled RTOS on the STM32F103 that fuses keypad and accelerometer input into a drivetrain model (engine speed, vehicle speed, gear, brake torque) at a bounded, schedulable CPU utilization.
2) A diagnosis of the end-to-end telemetry pipeline that identified UART baud rate, plotting-library overhead, and the lack of concurrency as the dominant contributors to a $\approx$60 s visualization latency.
3) A multithreaded, PyQtGraph-based visualization pipeline on the Raspberry Pi that reduces this latency to near real time.
4) A Tkinter-based bidirectional control dashboard that allows an operator to command torque, brake, and direction from the host back to the microcontroller, in addition to the on-board keypad interface.

B. Paper Organization

Section II reviews the embedded-systems background relevant to the design choices made in this project. Section III describes the hardware and firmware architecture, including the RTOS task set and its schedulability. Section IV details the real-time visualization bottleneck and its resolution. Section V describes the bidirectional control interface and validated use cases. Section VI reports results, Section VII concludes, and Section VIII outlines future work.

II. Background

A. Embedded Systems and RTOS

An embedded system combines hardware and software dedicated to a specific task, typically operating autonomously within a larger system. A real-time operating system (RTOS) extends this model with timing guarantees: tasks are scheduled such that critical deadlines are met deterministically, which is essential in domains such as automotive control, industrial automation, and — as in this work — agricultural machinery, where a delayed torque or brake response has direct safety and efficiency consequences. Relative to a bare-metal design, an RTOS trades a modest increase in memory footprint and design complexity for modular, priority-driven task management and predictable interrupt handling, which is what allows the accelerometer-sampling, keypad-reading, display, and communication duties in this project to coexist on a single core without ad hoc polling.

B. Embedded Linux on the Host

The Raspberry Pi runs a general-purpose embedded Linux distribution (Raspberry Pi OS), which trades the deterministic timing of an RTOS for flexibility, a mature driver ecosystem, and access to high-level languages and libraries. This makes it well suited to the host-side responsibilities in this project — serial data ingestion, real-time plotting, and GUI-driven control — but, as discussed in Section IV, it also means that latency on this side of the link is governed by software efficiency (library choice, threading model) rather than by scheduling guarantees.

III. System Architecture

A. Hardware

The system is built around two compute nodes connected by a serial link. On the embedded side, an STM32F103RBT6 (ARM Cortex-M3) reads a linear potentiometer standing in for the accelerometer-based throttle channel, a second potentiometer for auxiliary sensing, and a 4×4 matrix keypad used to command direction and braking. State is displayed locally on a 16×2 character LCD and transmitted over USART. An FT232RL FTDI USB-to-serial bridge exposes this UART link to the Raspberry Pi 3 Model B+, which hosts the visualization and control software. Fig. 1 shows the complete schematic.

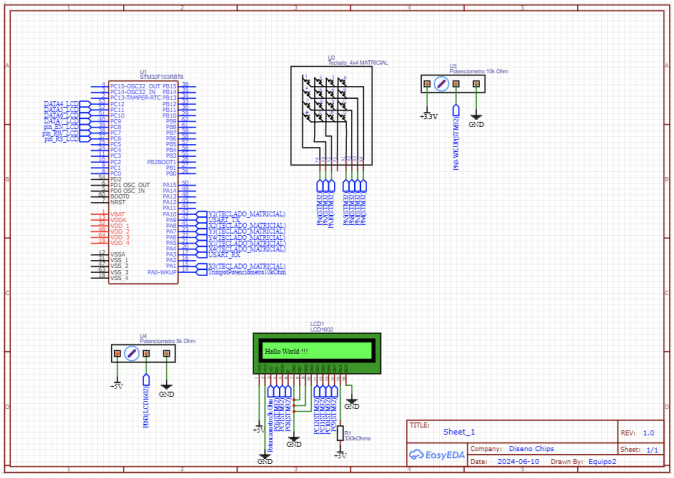

Fig. 1. Complete hardware schematic: STM32F103RBT6 MCU, 4×4 matrix keypad, throttle and auxiliary potentiometers, and 16×2 LCD.

B. Firmware and RTOS Task Set

The STM32F103 firmware is developed in C using STM32CubeIDE and CMSIS, and structured as four periodic RTOS tasks that communicate through a shared drivetrain model (`EngTrModel`) rather than direct coupling, so that any task can update or consume vehicle state without blocking the others:

- **Task 1** (`Task1_Init`): initializes and polls the keypad (`KEYPAD_Init`, `KEYPAD_ReadKey`), maps key presses to a commanded direction, and scales key input into a brake-torque value.
- **Task 2** (`Task2_Init`): initializes and samples the throttle ADC channel (`USER_ADC1_Init`, `USER_ADC1_Read`), scaling the accelerometer/potentiometer reading into the drivetrain model's throttle input.
- **Task 3** (`Task3_Init`): refreshes the LCD (`LCD_Init`, `LCD_Clear`, `LCD_Set_Cursor`, `LCD_Put_Str`, `LCD_Put_Num`) with engine speed, direction, and vehicle speed.
- **Task 4** (`Task4_Init`): initializes USART1 and transmits (`USER_USART1_Init`, `USER_USART1_Transmit`) the updated drivetrain values to the Raspberry Pi.

Fig. 2 shows the resulting data-flow graph: keypad and potentiometer inputs feed Tasks 1 and 2, which update the shared model; Task 3 renders it locally on the LCD, and Task 4 forwards it to the host over UART.

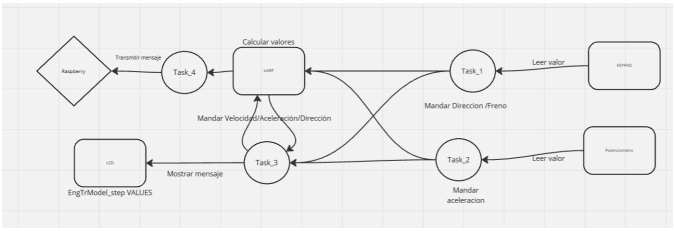

Fig. 2. RTOS task interaction graph. Tasks 1 and 2 ingest keypad and potentiometer input into the shared drivetrain model; Task 3 drives the LCD and Task 4 transmits state to the Raspberry Pi over UART.

Table I summarizes the periodic task set used for schedulability analysis, and Table II maps each task to its function and dependencies.

Under rate-monotonic scheduling, priorities are assigned inversely to period, and the task set is schedulable if total utilization $U = \sum_i C_i/T_i$ stays below the classical Liu–Layland bound for n tasks. With the periods and worst-case

TABLE I
PERIODIC TASK SET (RATE-MONOTONIC)

Task ID	Period (ms)	Duration (ms)	Priority
4	50	1	Highest
5	50	2	
6	118	48	
7	250	72	Lowest

durations in Table I:

$$U = \frac{1}{50} + \frac{2}{50} + \frac{48}{118} + \frac{72}{250} \approx 0.7548,$$

i.e., a measured CPU utilization of 75.48%, comfortably schedulable and leaving headroom for interrupt handling and future tasks.

TABLE II
TASK SET SUMMARY

Task	Description	Key dependencies
Task 1	Reads keypad, sets direction, scales brake torque	`KEYPAD_*`
Task 2	Reads throttle ADC, scales value into drivetrain model	`USER_ADC1_*`
Task 3	Refreshes LCD with engine/vehicle speed, direction	`LCD_*`
Task 4	Transmits drivetrain state over USART1	`USER_USART1_*`

C. Host-Side Software Stack

The Raspberry Pi runs Raspberry Pi OS, flashed via Raspberry Pi Imager, with development carried out in Visual Studio Code and Thonny and serial bring-up verified with PuTTY. The host application is written in Python and is responsible for (i) ingesting the UART stream forwarded by the FTDI bridge, (ii) rendering it in real time, and (iii) issuing control commands back to the STM32.

IV. REAL-TIME VISUALIZATION OPTIMIZATION

A. Problem

Real-time monitoring is central to this project's value proposition: for time-sensitive field operations such as planting and harvesting, an operator or supervisory system needs an immediate, accurate picture of engine and vehicle state. In the initial implementation, however, the Matplotlib-based plotting pipeline exhibited approximately one minute of end-to-end latency between a change at the STM32 and its appearance on the Raspberry Pi display, which made the dashboard unusable for its intended purpose.

B. Root-Cause Analysis

Three compounding factors were identified:

1) **Low UART baud rate**, which capped the rate at which drivetrain samples could reach the host regardless of downstream processing speed.
2) **Single-threaded, blocking pipeline**: serial acquisition, data processing, and plot redraw executed sequentially on the Raspberry Pi's constrained CPU, so each stage stalled on the previous one.
3) **Matplotlib overhead**: Matplotlib's rendering backend is optimized for static or moderately interactive figures rather than high-frequency redraws, and its per-frame cost dominated the pipeline once combined with the two issues above.

C. Solution

The pipeline was rebuilt around three changes. First, the UART baud rate was raised to remove the transmission-side bottleneck. Second, the plotting backend was switched from Matplotlib to PyQtGraph, which is designed for high-frequency real-time plotting with substantially lower per-frame CPU cost. Third, the application was restructured with dedicated threads so that serial acquisition, data processing, and GUI rendering execute concurrently rather than sequentially, preventing any single stage from blocking the others. Together, these changes reduced the observed latency from roughly one minute to near real time, restoring the dashboard's usefulness for live monitoring.

V. BIDIRECTIONAL CONTROL INTERFACE

In addition to passive monitoring, the Raspberry Pi hosts a Tkinter GUI (Fig. 3) that lets an operator issue commands back to the STM32, mirroring the physical keypad interface on the embedded side. Two APIs are combined: PyQtGraph renders the live telemetry stream from the STM32, while Tkinter implements the control widgets.

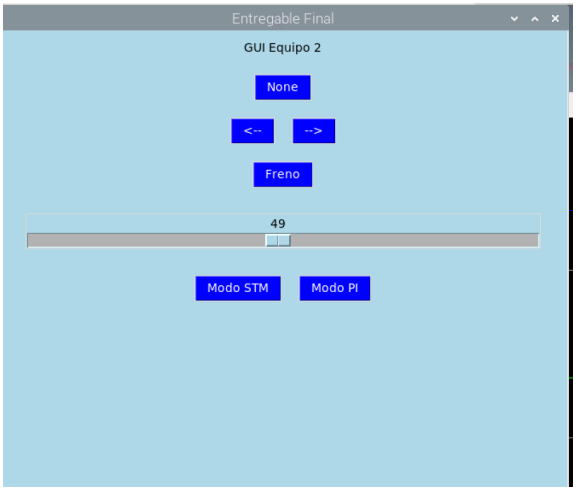

Fig. 3. Tkinter control dashboard running on the Raspberry Pi: direction toggle, brake button, throttle slider, and STM/PI mode selector.

The dashboard exposes:

- A **direction toggle**, which latches the last commanded direction (left/right) and re-sends it continuously until changed, mirroring the keypad's persistent-direction behavior.

- A **brake button** for commanded braking.
- A **throttle slider**, functioning as a software potentiometer for commanded speed.
- A **Mode STM / Mode PI** switch: in *Mode STM*, the host only ingests and plots values streamed from the STM32; in *Mode PI*, the host instead transmits the slider- and button-derived values to the STM32, closing the control loop from the Raspberry Pi side.

This design was validated against both directions of control: torque increase/decrease, braking, and left/right steering commanded from the physical STM32 keypad, and the same set of actions commanded from the Raspberry Pi GUI while the STM32 responded to the incoming values, confirming bidirectional consistency of the drivetrain model on both ends of the UART link.

VI. Results

The RTOS task set is schedulable under rate-monotonic priority assignment at 75.48% measured CPU utilization (Table I), leaving margin for interrupt servicing without missing deadlines. On the host side, replacing Matplotlib with a multithreaded PyQtGraph pipeline and raising the UART baud rate reduced end-to-end visualization latency from approximately 60 s to near real time, which was the primary usability blocker identified during testing. The bidirectional Tkinter interface was exercised through matched use-case pairs (throttle up/down, brake, steer left/right) issued from both the keypad and the GUI, with the drivetrain state on the LCD and on the dashboard remaining consistent in both control directions.

VII. Conclusion

This work demonstrates a low-cost, dual-processor architecture for real-time monitoring and control of agricultural machinery, combining the deterministic task scheduling of an STM32F103 RTOS with the processing flexibility of a Raspberry Pi running embedded Linux. Beyond the initial sensing and display pipeline, the key engineering contribution was diagnosing and resolving a visualization pipeline that, if left unaddressed, would have made the system unsuitable for the time-critical field operations it targets: moving from a blocking, Matplotlib-based pipeline to a multithreaded PyQtGraph pipeline, together with a corrected baud rate, closed the gap between a one-minute-latency prototype and a usable real-time dashboard. The addition of a bidirectional Tkinter control surface further extends the system from passive telemetry to active remote actuation, positioning it as a building block toward more autonomous precision-agriculture platforms.

VIII. Future Work

Planned extensions include: replacing the wired UART/FTDI link with a wireless channel (e.g., Bluetooth or Wi-Fi) for untethered operation; applying machine-learning-based predictive maintenance on the Raspberry Pi to flag mechanical anomalies from streamed drivetrain data; developing a more accessible mobile or web interface for operators with varying levels of technical experience; and closing the loop with a predictive-maintenance scheduler that uses continuously logged operational data to reduce tractor downtime.

Acknowledgment

The authors thank John Deere for sponsoring this project, and Professors José Isabel Gómez Quiñones and Rodolfo Anaya Zamora for their guidance throughout its development at Tecnológico de Monterrey.

References

[1] B. JL, "Sistema embebido," *Electrónica Online*, Mar. 2024. [Online]. Available: https://electronicaonline.net/electronica/sistemas-embebidos/

[2] E. R. Moraguez, ""Qué es un Microcontrolador: cómo funciona y para qué sirve?"," *LovTechnology*, Mar. 2023. [Online]. Available: https://lovtechnology.com/que-es-un-microcontrolador-como-funciona-y-para-que-sirve/

[3] Isaac, "RTOS: qué es un sistema operativo de tiempo real," *Hardware Libre*, Jan. 2022. [Online]. Available: https://www.hwlibre.com/rtos/

[4] Trbl, "LINUX Embebido: Qué es, cómo funciona y para qué se usa," *TRBL Services*, Jan. 2024. [Online]. Available: https://trbl-services.eu/blog-linux-embebido-que-es/

[5] "Bash Script - Guía completa con ejemplos," *Blog HostingTG*, May 2024. [Online]. Available: https://www.hostingtg.com/blog/bash-script/

[6] STMicroelectronics, "STMicroelectronics: Our technology starts with you." [Online]. Available: https://www.st.com/content/st_com/en.html

[7] "eLinux.org." [Online]. Available: https://elinux.org/Main_Page

[8] "The Yocto Project," May 2024. [Online]. Available: https://www.yoctoproject.org/

[9] Raspberry Pi Foundation, "Teach, learn, and make with the Raspberry Pi Foundation." [Online]. Available: https://www.raspberrypi.org/